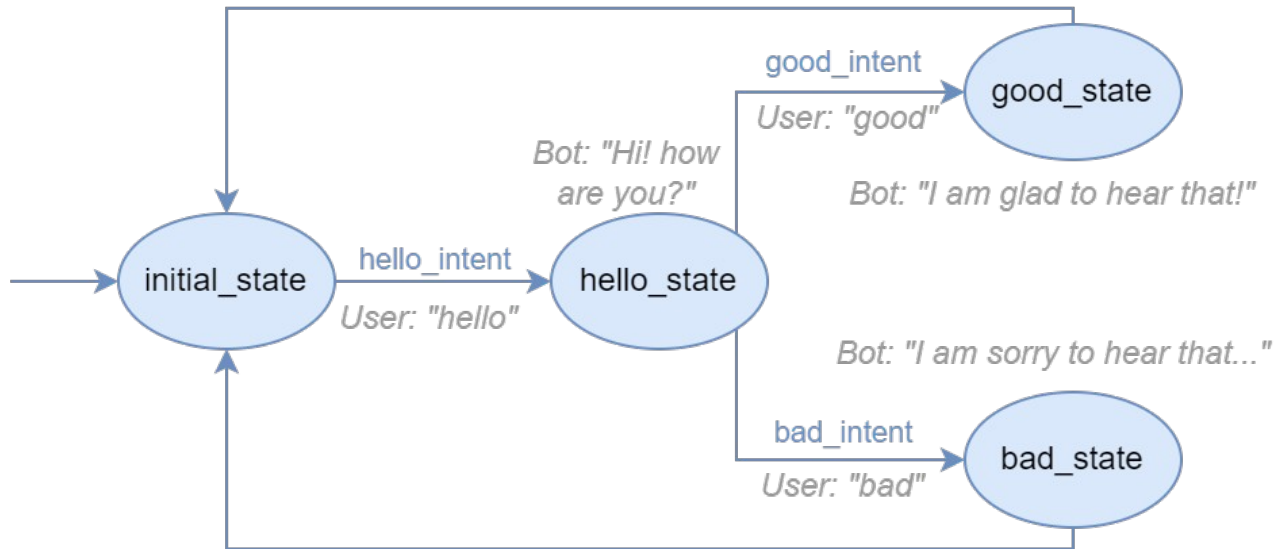


# Core

The core package defines and implements the basic components of an agent architecture. BAF relies on [State Machines](#) to define the agent execution logic.



## States

An agent is composed by a set of states. The user can navigate through the agent states by triggering the transitions that move the user's current state in the agent to another one. Creating a new state is really simple:

```
agent = Agent('example_agent')
...
example_state = agent.new_state('example_state')
```

The agent must have 1 (and only 1) initial state, which is the starting point of the agent. To create an initial state, simply set the following attribute:

```
initial_state = agent.new_state('initial_state', initial=True)
```

## State body

Agent states have a Python function attached, called *body*, which is executed when the user moves to them. Let's see an example body:

```
def example_body(session: Session):
    session.reply('Hi! How are you?')
```

This body simply sends a message to the user. As you can guess, you can do anything within a body function, but always be careful to avoid errors.

Note

All exceptions produced in body functions are properly handled by the agent, so it will not crash the agent application. Nevertheless, we always recommend to be cautious and test your agents.

Note

You can create states with no body function. These states will then have an empty body function that does nothing.

Note

The initial state's body function is automatically run when a user establishes connection with the agent.

The body function must have 1 (and only 1) parameter: the user [session](#). There, the agent will read/write user-related data (i.e private data). If your agent needs to read/write global or shared data among all users, you can define any variable outside the body and use it inside.

Now, we must assign this body function to our state:

```
example_state.set_body(example_body)
```

## State fallback body

States have another body function that is intended to be run whenever the state fails when transitioning to another state. A typical scenario is when the user sends a message to the agent, but the agent does not recognize the user intent, and therefore, runs the fallback body of the user's current state. The process is the same as with regular body functions:

```
def example_fallback_body(session: Session):  
    session.reply(session, "Sorry, I didn't understand you")  
  
example_state.set_fallback_body(example_fallback_body)
```

Note

By default, all states have a fallback body that simply replies to the user "Sorry, I didn't get it"

There is a way to assign a custom fallback body to all agent states at the same time (replacing all previously assigned fallback bodies). You need to do it through the agent object:

```
agent.set_global_fallback_body(example_fallback_body)
```

Note that if you create a state after that, it will not have your global fallback assigned.

## State transitions

See the [transitions](#) guide to learn about the different kinds of transitions and how to define them.

## Global States

Global States are a special kind of states.

As opposed to normal states, which are only accessible during certain parts of the agent conversation, global-states are accessible from anywhere. In that sense, we could also talk about global-state-components, which consist of a series of states that is triggered once the first state is accessed.

Let's visualize this by taking a look at a simplified greetings agent example!

Now let's say we would want to add a "help" state, which should help the user in case they don't know how to proceed or want additional information. To avoid adding the single transitions to each state, it is possible to define the "help" state as a global state. For that purpose, we first define the "help" state as we would any other state:

```
help_state = agent.new_state('help_state')
```

Which results in the following state machine:

Currently, it is only possible to trigger a global-state by specifying an Intent that should trigger the state. In our case, let's say we prepared an Intent called "**help\_intent**". Now we need to specify that "**help\_state**" is a global-state:

```
help_state.set_global(help_intent)
```

What happens now, is that "**help\_state**" will be seen as a global-state by our agent. This results in the necessary transitions being automatically added by the agent.

Note that, regarding the agent's actions during the "**help\_state**", one can define it as with any other state by setting the body.

If a user would trigger the "**help\_state**", then the agent will move to the "**help\_state**", act as specified in the body, and then jump back to the state the user found themselves in when triggering the global-state. An example could be the following:

Greetings agent diagram with transitions to and from help\_state

Of course, one can add more transitions to a global-state. Here, we could talk about a global-state-component, which consists of a series of states only accessible once the initial state in the respective global-component has been triggered.

Extending the initial "**help\_state**" works just as with any other state by creating new states and adding the necessary transitions. E.g. if we would like to add a state that should be entered if the user thanks the agent for helping them, then we add the following lines:

```
youre_welcome_state = agent.new_state('youre_welcome_state')
...
help_state.when_intent_matched(thanks_intent).go_to(youre_welcome_state)
```

We could now see the states "**help\_state**" and "**youre\_welcome\_state**" as a global-state-component. Note that, the final transition back to the original state always gets added to the final state of the global-state-component.

Warning

Currently, only linear state sequences are supported for global-state-components. Thus, branching in global-states will work arbitrarily

## Intents

Intents define the *intentions* or *goals* the user can express to an agent. In this article we review how to create intents with BAF.

### Simple intents

First of all, we need to create an agent and some states:

```
agent = Agent('example_agent')
...
initial_state = agent.new_state('initial_state')
handle_hello_state = agent.new_state('handle_hello_state')
```

Let's say we want to say hello to the agent. For that, we need an intent that defines how do "hello" messages look like. To do that, we create an intent writing some **training sentences** as example messages we expect the agent to understand in a "hello" message:

```
hello_intent = agent.new_intent('hello_intent', [
    'hello',
    'hi'
])
```

Once we have defined our intent, we need to add the proper [transitions](#) to explicitly define where this intent is expected to be received and where the user is expected to move once the intent is received:

```
initial_state.when_intent_matched(hello_intent).go_to(handle_hello_state)
```

When the user is in `initial_state`, the agent is waiting for a user message. Once the user sends a message, the agent will classify it into one of the possible incoming intents in `initial_state` (here, only `hello_intent` can be recognized). If the message's intent is `hello_intent`, the user will move to `handle_hello_state`, as we have defined it.

### Intents with parameters

Intents can have embedded [parameters](#). This means that we can extract data from a user message. At the same time, this can help reducing the number of intents in an agent.

We are going to create an intent to ask about the weather in a city.

```
weather_intent = agent.new_intent('weather_intent', [
    'what is the weather in CITY?',
    'tell me the weather in CITY',
    'weather in CITY'
])
```

As you can guess, the *CITY* fragment of the training sentences is going to host a parameter where the user can say any city. Defining the intent parameter is really simple:

```
weather_intent.parameter('city1', 'CITY', city_entity)
```

Intent parameters have a **name**, a **fragment** that indicates the position of the parameter in the intent's training sentences and an **entity** that defines the values that can be matched in the parameter.

See the [entities](#) guide to learn about them.

Tip

You can also add a list of parameters directly in the intent creation:

```
weather_intent = agent.new_intent('weather_intent', training_sentences,
parameters)
```

Then, we can create a transition that is triggered when the user intent matches `weather_intent`:

```
initial_state.when_intent_matched(weather_intent).go_to(handle_weather_state)
```

## Intent descriptions

If you want to use the [LLM Intent Classifier](#), you can create an intent without training sentences. Instead, you just would need to provide a description for that intent, although you can still add training sentences to it. This is an example intent with a description:

```
help_intent = agent.new_intent(
    name='help_intent',
    description='The user needs help to be able to use the agent properly or to find some infor
```

## Getting the intent prediction results

Within a [State body](#) we can access the latest predicted intent from the user [session](#).

Warning

The predicted intent is only available when the session event is a [ReceiveTextEvent](#)

Let's see it with an example body function.

(here we specify the type of each object, although it is not necessary to do it)

```
def handle_weather_body(session: Session):
    # The predicted intent is stored in the last received event
    event: ReceiveTextEvent = session.event
    prediction: IntentClassifierPrediction = event.predicted_intent
    # We can get the intent object:
    intent: Intent = prediction.intent
    # We can get the score assigned to this intent by the intent classifier:
    score: float = prediction.score
    # We can get the message sent by the user:
    matched_sentence: str = prediction.matched_sentence
    # We can get the list of all matched parameters:
```

```

matched_parameters: list[MatchedParameter] = prediction.matched_parameters
# We can get a specific parameter by its name:
city = prediction.get_parameter('city1')

if city.value is None: # Sometimes the intent can be recognized, but not the
parameters
    session.reply("Sorry, I didn't get the city")
else:
    # Here we would call some API to get the temperature of the city
    temperature = some_service.get_temperature(city)
    session.reply(f"The weather in {city.value} is {temperature}°C")
    if temperature < 15:
        session.reply('🥶')
    else:
        session.reply('🥵')

```

# Entities

Entities are used to specify the type of information to extract from user inputs. These entities are embedded in intent parameters.

## Custom entities

Let's see how to create your own entities for your agent.

```

agent = Agent('weather_agent')
...
city_entity = agent.new_entity('city_entity', entries={
    'Barcelona': ['BCN', 'barna'],
    'Luxembourg': ['LUX'],
})

```

Entities have a set of **entries**. Each entity entry is composed by a **value** (e.g. 'Barcelona') and a list of (optional) **synonyms** of the value (useful for values that can be called in different ways, but refer to the same concept). In this example entity, you could add all the cities you want the agent to be able to recognise. In a real use case, you could also get a complete list of cities in a country and create an entity with that entries.

In [Intents with parameters](#), we explain how to embed entities in intent parameters.

## Entity descriptions

If you want to use the [LLM Intent Classifier](#), you can create an entity without values. Instead, you just would need to provide a description for that entity, although you can still add values to it. This is an example intent with a description:

```

email_entity = agent.new_entity(
    name='email_entity',
    description='An entity containing email addresses',
)

```

In this example, the LLM Intent Classifier will be able to recognize and extract email addresses from user messages.

## Base entities

In BESSER Agentic Framework, we include a set of predefined entities that can be used in any intent parameter like custom entities. The difference is that you don't need to define them. We do it for you!

The following list describes all currently implemented base entities:

| Name                            | Description                           | Example values                                                                                             |
|---------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------|
| <a href="#">number_entity</a>   | Matches a number                      | 3, 3.5, -3.5, minus three                                                                                  |
| <a href="#">datetime_entity</a> | Matches a date, a time or a date-time | 2023-09-14, 08:30:00, 1985-09-25 17:45:30, today, tomorrow at 5pm, in five years and 2 weeks, 3 months ago |

Let's see how to use base entities with an example:

```
from baf.library.entity.base_entities import number_entity
...
agent.add_entity(number_entity)

age_intent = agent.new_intent('age_intent', [
    'I am NUM years old',
    'My age is NUM
'])
age_intent.parameter('age', 'NUM', number_entity)
...
def age_body(session: Session):
    event: ReceiveTextEvent = session.event
    prediction: IntentClassifierPrediction = event.predicted_intent
    age = predicted_intent.get_parameter('age').value
    session.set('age', age) # Save the age in the user session
    session.reply('Thanks for the information')
...
```

## Events

An agent can receive [events](#) through its platforms.

An agent can define transitions from one state to another based on the reception of specific events.

Note

The [Transitions](#) wiki page explains how to define event-based transitions in agents.

All events contain the following information:

- **name:** name of the event

- **session\_id**: the id of the session the event was sent to. If an event has been broadcasted to all agent sessions, this attribute is empty
- **timestamp**: the timestamp indicating the event reception instant

Then, **each event implementation may have different attributes**.

You can access the received event from the user session (i.e., from [state bodies](#) or [transition conditions](#))

The session always contains the last received event (the one that triggered the transition to the state you are reading the event from)

```
def example_body(session: Session):
    event: Event = session.event
    # Depending on the type of event, we can access specific information
    if isinstance(event, ReceiveTextEvent):
        message: str = event.message
        intent_prediction: IntentClassifierPrediction = event.predicted_intent
        human: bool = event.human # Whether the message was sent by a human or
not
    if isinstance(event, ReceiveJSONEvent):
        # If the JSON contained a 'message' field:
        message: str = event.message
        intent_prediction: IntentClassifierPrediction = event.predicted_intent
        json: dict = event.json # The full JSON payload
        human: bool = event.human # Whether the message was sent by a human or
not
    if isinstance(event, ReceiveFileEvent):
        file: File = event.file
        human: bool = event.human
    if isinstance(event, GitHubEvent) or isinstance(event, GitLabEvent):
        category = event.category
        action = event.action
        payload = event.payload
```

Next, we introduce all available events in BAF.

## Base events

This set of events are used to receive data, usually in the form of text, JSON or file messages.

Currently, [WebSocket platform](#) and [Telegram platform](#) use these events to send data to the agents.

For example, when a user sends a text message to an agent through the WebSocket platform, the platform packs it in a `ReceiveTextEvent` so the agent can use it to trigger the appropriate transition to another state.

Important

All events described in [base\\_events](#) API documentation.

## GitHub events

You can use the [GitHub platform](#) in an agent to receive events from GitHub.

Important

All events described in [github\\_webhooks\\_events](#) API documentation.



## GitLab events

You can use the [GitLab platform](#) in an agent to receive events from GitLab.

Important

All events described in [gitlab webhooks events](#) API documentation.

## Transitions

Transitions define the connections between agent [states](#) and how/when are they triggered.

A transition is a rule that says that the agent must move from a source state to a destination state when some event and/or condition occurs.

Warning

Transitions' order matters! The order in which a state's transitions are created is the order in which they will be evaluated.

In this section, we will explain how to create transitions and the various methods available for creating predefined transitions in an easy way.

Note

Transitions of a session current state are evaluated periodically every N seconds. You can set N as an agent property. See the [agent properties](#).

Whenever a session receives an incoming event, transition evaluation is also run.

Let's say we have the following agent:

```
agent = Agent('example_agent')

state1 = agent.new_state('state1', initial=True)
state2 = agent.new_state('state2')
state3 = agent.new_state('state3')
...
```

## Transition Builder

To help us define a full transition, we use a [TransitionBuilder](#) object that will add each element to the transition for us.

Essentially, a Transition Builder has 3 elements:

- **Source:** the source state of the transition we want to build
- **Event:** the event that should trigger the transition (**optional**)
- **Condition:** the condition that should trigger the transition (**optional**)

If both event and condition are specified, both need to be fulfilled in order to trigger the transition. If none of them is specified, the transition will always be triggered without restrictions.

Fortunately, BAF provides you with functions that abstract you of these concepts.

Let's see an example transition that should be triggered when a `ReceiveMessageEvent` is sent to the agent from state1:

```
from baf.library.transition.events.base_events import ReceiveMessageEvent
...
```

```
event = ReceiveMessageEvent()
```

```
transition_builder: TransitionBuilder = state1.when_event(event)
```

This will create an (incomplete) transition builder. We still need to specify the destination state:

```
transition_builder.go_to(state2)
```

That's it! The transition has been already registered into the agent.

**In practice, you will define both the event and the destination state in the same code line:**

```
state1.when_event(event).go_to(state2)
```

Note

You can leave the event argument empty to match any incoming event (a **“wildcard”** event):

```
state1.when_event().go_to(state2)
```

If you want to **add a condition** to the transition, you can do so in the same line of code:

```
state1.when_event(event).with_condition(some_condition).go_to(state2)
```

You can also **add multiple conditions** to the same transition (so all of them will be merged into a single one)

```
state1.when_event(event).with_condition(condition1).with_condition(condition2).go_to(state2)
```

It is also possible to define a transition **only with condition (without event)**:

```
state1.when_condition(some_condition).go_to(state2)
```

Let's quickly explain events and conditions.

## Events

An agent can receive events through its platforms. An agent can define transitions from one state to another based on the **reception of specific events**.

In the [Events wiki](#) page you will find more details about all available events.

## Conditions

A condition is simply **a Python function that returns a boolean value** (i.e., True or False). When returning true, we say the condition is satisfied (which is the requirement of a condition-based transition to be triggered).

A condition function only has an **argument of type Session** (i.e., the session of the current user, similar to the session parameter of a state body). This way, conditions can read/write data on the user session.

Let's see it with an example. Let's assume that some agent states update a session variable called 'count'. We can define a condition that will be satisfied when 'count' is greater than 3:

```
def my_condition1(session: Session):  
    return session.get('count') > 3
```

Now, we can add our condition to a transition:

```
state1.when_condition(my_condition1).go_to(state2)
```

Optionally, a condition can have a **second argument of type dictionary, where you can store some parameters**. This way, we can reuse the same condition function with different data:

```
def my_condition2(session: Session, params: dict):  
    return session.get('count') > params['target_count']  
  
state1.when_condition(my_condition2, params={'target_count': 3}).go_to(state2)  
state1.when_condition(my_condition2, params={'target_count': 5}).go_to(state3)
```

Finally, let's see another way to implement equivalent conditions with **lambda functions**.

```
my_condition1 = lambda session: session.get('count') >= 3  
my_condition2 = lambda session, params: session.get('count') >= params['target']
```

We can also define **time-based conditions**. Let's see an example:

```
from datetime import datetime  
...  
  
def time_condition(session: Session, params: dict):  
    # Returns true if the current date is after a target date  
    return datetime.now() > params['target_date']  
  
state1.when_condition(time_condition, params={'target_date': datetime(2025, 7,  
3)}).go_to(state2)
```

## Built-in transitions

BAF provides some default built-in transitions you can use in your agents without having to manually implement events or condition.

Here we describe each of them.

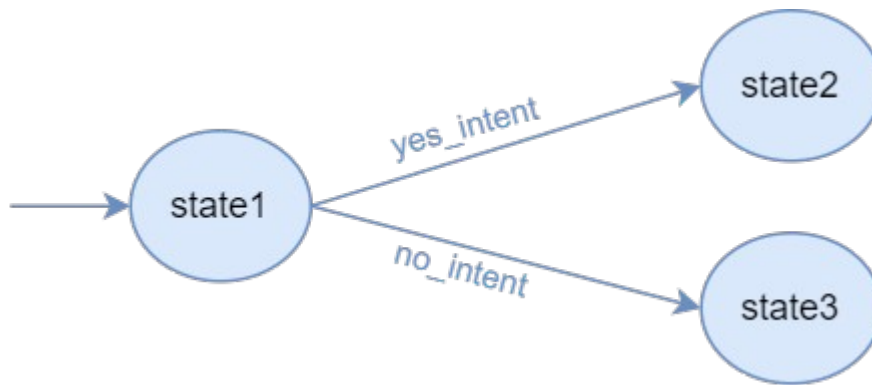
### Intent Matching

When the user sends a message to the agent, it gets the user intent and uses it to decide which state to move to. When the user intent matches with a specified transition intent, the agent moves to that transition's destination state.

It uses the [ReceiveTextEvent](#) and [IntentMatcher](#) condition.

```
state1.when_intent_matched(yes_intent).go_to(state2)  
state2.when_intent_matched(no_intent).go_to(state3)
```

Visually, this would be the agent architecture:



You can also define where to go when none of the previous intents is matched:

```
state1.when_no_intent_matched().go_to(state4)
```

Note **that there is the possibility that the agent cannot transition to any state**. For example, if we do not define the last transition, when neither `yes_intent` nor `no_intent` are matched the agent would not know where to move. In that scenario, the agent would run the `state1`'s fallback body, without moving to another state (see [State fallback body](#) for more info). Thus, it is up to the agent creator to choose whether, in case no intent is matched, a transition to another state takes place or not.

## Session variables

We can define transitions that take a session variable and applies some operator to compare it with a target value. In the previous section we saw how to manually create such conditions, but here you will see how to do it with the built-in transitions.

This transition uses the [VariableOperationMatcher](#) condition (and no event).

```
import operator # You can import a set of predefined operations from here (or
define your own)
state1.when_variable_matches_operation('money', operator.lt,
10000).go_to(state2)
```

The operator is a Python function that takes the session variable and the target value as arguments and returns a boolean value.

One must be cautious when defining these transitions. Following the previous example, when this transition is evaluated, there must be a 'money' variable in the session (that has to be added in some state body), otherwise this transition will never be triggered.

## File reception

It is also possible to trigger a transition in case a file is sent to the agent

It uses the [ReceiveFileEvent](#).

```
state1.when_file_received(allowed_types=["application/pdf",
"image/png"]).go_to(state2)
```

Note that it is also possible to define a list of allowed file types, so we can impose restrictions to what can be sent by users to avoid unwanted file types to be processed. Don't add this parameter if you want to receive any kind of file.

## Automatic transitions

Another simple but useful kind of transition is the automatic transition. When a state finishes the execution of its body, if it has an automatic transition the agent will always move to the transition's destination.

This is really useful when, after a sequence of states, we want to automatically return to the starting point of the conversation.

This transition has no event nor condition (therefore, being always satisfied)

```
state1.go_to(state2)
```

### Warning

The automatic transition cannot be combined with other transitions in the same state.

# Sessions

An agent is designed once, and when it is deployed there is not a copy of it for each user. There is a single agent instance. Therefore, we need a way to organize each user context. Of course, at the same time there can be multiple users, each of them in a different agent state, with different values for some variables, etc. To handle this, we introduce the Session object.

**A session is an object assigned to each user where it is stored private data.**

If you dig into the BAF's guts, you will see that some functions receive a Session object as a parameter. That is because these functions can modify the user session. Most of the session processes are done internally, so you don't need to worry about it.

You are able to manipulate the user session in 2 places:

- State body functions (see the [State body](#) documentation)

```
def example_body(session: Session):  
    # Your code here
```

- Transition conditions (see the [Conditions](#) documentation)

```
def example_condition(session: Session, params: dict):  
    # Your code here
```

We will stick to the body use case, although the same can be done in transition conditions.

To understand how this internally works, we must know that the body function is defined once, and assigned to a state. Whenever a user moves to a state, this function is executed by the agent, but taking as argument the user session. This way the agent can read and write user-specific data stored there and it cannot do it with other users data.

Let's see the different things we can do with a user session.

(here we specify the type of each object, although it is not necessary to do it)

```
def example_body(session: Session):
    # We can get the received event:
    event: Event = session.event # Each type of event has different information
    we can access
    # We can get the session id:
    session_id: str = session.id
    # We can send a message to the user:
    session.reply('Hello!')
    # We can get the chat history (the 'n' last messages):
    chat_history: list[Message] = session.get_chat_history(n=5)
    # We can set (store) a variable:
    session.set('age', 30)
    # We can get a variable (the return type can be any type):
    age: int = session.get('age')
    # We can delete a variable:
    session.delete('age')
    # We can send a message to another agent
    session.send_message_to_websocket('ws://localhost:5000', 'Hello!')
    # We can run RAG
    session.run_rag('your question')
```

Note

See [Events](#) to learn about Events

Note

To access the chat history, you need to set up the [Monitoring Database](#)

Note

See [Persisting sessions across restarts](#) to learn about persisting sessions across restarts

Note

See [RAG](#) to learn about Retrieval Augmented Generation

## Processors

The *Processor* class defines an abstract base class for processing user or agent messages within the system. It serves as a template to implement processors that can modify messages before they are handled by other components.

## Overview

Processors are used to define custom processing steps for messages exchanged between users and the agent. They can be applied to either user messages, agent messages, or both.

To implement a custom processor, inherit from the *Processor* class and override the *process* method.

# Creating a custom Processor

Here is an example of how to create a custom processor that processes both user and agent messages:

```
from baf.core.agent import Agent
from baf.core.session import Session
from baf.processors.processor import Processor

class MyCustomProcessor(Processor):
    def __init__(self, agent):
        super().__init__(agent=agent, user_messages=True, agent_messages=True)

    def process(self, session: 'Session', message: str) -> str:
        # Custom processing logic goes here
        processed_message = message.lower() # Example: convert message to lowercase
        return processed_message

# Usage
agent = Agent('greetings_agent')
# Initialise processor
my_processor = MyCustomProcessor(agent)
```

The example demonstrates the implementation of a *MyCustomProcessor* that converts both the user's messages to the agent and the agent's responses to lowercase.

Note

- A processor can also store some information in the user session, see [LanguageDetectionProcessor](#) as an example.
- A processor can use other agent components. For instance, [LLMs](#) already defined in the agent can be accessed within the processor. If you define a parameter *llm\_name* in your processor, it could be accessed within the process method as follows: *llm = self.agent.nlp\_engine.\_llms[self.llm\_name]*

## Methods

*Processor* defines the following methods:

- **`__init__(self, user_messages: bool = False, agent_messages: bool = False)`**

Initializes a new *Processor* instance.

Parameters:

- **`agent`** (*Agent*): The agent the processor belongs to.
- **`user_messages`** (*bool*): Whether the processor should be applied to user messages. Default is *False*.
- **`agent_messages`** (*bool*): Whether the processor should be applied to agent messages. Default is *False*.

Raises:

- **ProcessorTargetUndefined:** If neither *user\_messages* nor *agent\_messages* is set to *True*.
- **process(self, session: 'Session', message: Any) -> Any**

Abstract method that processes a message. This method should be overridden in subclasses to define custom processing logic.

Parameters:

- **session** (*Session*): The current session object, containing the state and context of the agent-user interaction.
- **message** (*Any*): The message to be processed.
- **Any:** The processed message. The return type can vary depending on the implementation.

Note

The process method MUST return the processed message, even if no changes were applied.

## Raising Exceptions

The following exceptions may be raised:

- **ProcessorTargetUndefined:** Raised if a *Processor* is created without specifying either *user\_messages* or *agent\_messages* as *True*.

## Available processors

This section contains a list of implemented processors.

### LanguageDetectionProcessor

The [LanguageDetectionProcessor](#) attempts to detect the language of given messages by using the [langdetect](#) library. When processed, the recognized language will be stored as a session variable in ISO 639-1 format and can be fetched with the following call:

```
session.get('detected_lang')
```

### UserAdaptationProcessor

The `UserAdaptationProcessor` attempts to adapt the agent's responses based on the user's profile. The user profile can be added using the following call:

```
processor.add_user_model(user_model)
```



# Files

The [File](#) class was added as a way to help with the definition of file objects that are both sent and received by users.

## File Object Definition

A file object consists of 3 attributes:

- name (str): The name of the file.
- type (str): The type of the file.
- base64 (str): The base64 representation of the file.

Yet, to create a file object using the constructor, 3 options are possible.

- Providing the file content as a base64 string
- Providing a file path
- Providing the raw file data, in bytes

With this, we want to allow users to choose the option that is easiest to them and take care of the necessary conversion. Thus, users can choose whether to set file\_base64, file\_path or file\_data.

## Receiving and Sending Files

In case a file transition was specified, once it takes place the file object will be available in the Session's event:

```
def body(session: Session):
    event: ReceiveFileEvent = session.event
    file: File = event.file
    file_name = file.name
    file_type = file.type
    file_base64_data = file.base64
    # further processing here
```

For sending files to the users, agent creators will first have to create a file object and use the following platform-specific function (not every platform might support it):

```
def body(session: Session):
    file = File(file_name="name", file_type="type", file_base64="file_base64")
    websocket_platform.reply_file(session=session, file=file)
    # OR
    telegram_platform.reply_file(session=session, file=file)
    # in case you want to add a caption to your file, you can also set the
    message parameter
    telegram_platform.reply_file(session=session, file=file, message="Your
    Message")
```